Gradewise

AI Assignment Reviewer with a Teacher-in-the-Loop

Technical Report

LangChain | LangGraph | LangSmith | Groq | FastAPI

Prepared by: **Anushay Ayat**
Program: AI and Data Science, SMIT

Abstract

This report documents Gradewise, a web application built from a teacher-supplied terminal script that reviews a student answer with a Groq-hosted language model. The original program is a three-node LangGraph workflow: an *evaluate* node scores the answer out of 10, and a conditional edge sends scores from 4 to 6 to a *teacher_review* node, while all other scores go to *finish*. The project preserves that design and extends it into a complete system: a layered FastAPI backend, a multi-page single-page frontend, SQLite storage, custom questions with weighted rubrics, a teacher approval queue, an analytics dashboard with AI-versus-teacher agreement, batch review, file upload, an answer-similarity check, teacher and student accounts, PDF and CSV export, automated tests and LangSmith tracing with teacher feedback attached to each run.

1. Introduction and objectives

Manual review of written answers is slow and varies between reviewers. A language model can give fast, structured feedback, but its judgement on borderline answers should not be final. The objectives were:

- Run and understand the teacher's original LangGraph workflow.
- Integrate LangSmith so that every review can be inspected as a trace.
- Provide a professional backend, frontend and folder structure.
- Add features that make the tool useful in practice: rubrics, history, a teacher queue, analytics and exports.
- Document the work with a report, a presentation and screenshots.

2. Analysis of the original assignment

The supplied project contained a single Python file, a README, a requirements list and an environment example. Its behaviour is:

- A hard-coded question and a single student answer are entered in the terminal.
- The **evaluate** node asks a Groq model for a score from 0 to 10, feedback and one improvement.
- A routing function sends scores from 4 to 6 to **teacher_review**; any other score goes to **finish**.
- LangSmith is enabled only through environment variables.

The application has no interface, no storage and a single question, which defines the extension work in this project.

3. Technology stack

Layer	Technology	Purpose
API	FastAPI, Uvicorn	REST endpoints and static file hosting
Validation	Pydantic v2	Typed request bodies and structured LLM output
LLM	LangChain + langchain-groq	Prompt template and structured output from a Groq model
Workflow	LangGraph	State graph with a conditional edge

Layer	Technology	Purpose
Tracing	LangSmith	Run traces, tags, metadata and feedback
Storage	SQLite (sqlite3)	Questions and reviews
Reports	ReportLab	Per-review PDF
Frontend	HTML, CSS, JavaScript	Single-page app with SVG charts and dark mode
Tests	pytest, httpx	API and routing tests

4. System architecture

The browser talks only to the FastAPI service. Route handlers call the compiled LangGraph, persist the outcome in SQLite and return JSON. The graph calls Groq through LangChain, and both the graph and the model call are traced to LangSmith when tracing is enabled.

```
Browser (SPA) --> FastAPI routes --> LangGraph --> LangChain --> Groq
                        |               |
                        v               v
                    SQLite         LangSmith (traces + feedback)
```

Backend modules

Module	Responsibility
app/main.py	Application factory, lifespan start-up, static frontend mount
app/config.py	Environment-based settings, demo-mode and tracing flags
app/schemas.py	Pydantic models for questions, rubrics, reviews and teacher decisions
app/db.py	SQLite schema, seeding with two sample questions, row mappers
app/api/routes.py	All REST endpoints
app/graph/	State, nodes and graph builder
app/services/llm.py	Groq chain and the offline demo grader
app/services/tracing.py	LangSmith configuration and teacher feedback
app/services/pdf.py	PDF result generator
app/services/auth.py	Password hashing, signed session cookies, role guards, login throttling
app/services/similarity.py	Word 3-gram Jaccard similarity
app/services/extract.py	Text extraction from PDF, DOCX, TXT, MD uploads

5. LangGraph workflow

The graph is **START** → **evaluate** → **route** → **(teacher_review | finish)** → **END**, the same shape as the teacher's design. The evaluate node is generalised from one score to a rubric.

Node	Input	Output
evaluate	question with rubric, student answer	per-criterion scores, total, score out of 10, feedback, improvement, mode

Node	Input	Output
route	score	teacher_review if REVIEW_MIN ≤ score ≤ REVIEW_MAX (default 4 to 6), otherwise finish
teacher_review	state	status "Flagged for teacher review", needs_teacher = true
finish	state	status "Feedback ready", needs_teacher = false

6. Rubric scoring

Each question has between one and eight criteria, each with a name, a maximum of 1 to 20 points and a short description. The model is asked for a structured object containing a score and comment per criterion, an overall feedback sentence and one improvement. The backend then:

- matches returned criteria to the rubric by name and gives 0 to any that are missing;
- clamps every score between 0 and that criterion's maximum, so a model error cannot inflate marks;
- normalises the total with *score = round(total / max_total × 10)* so the teacher's 4 to 6 rule keeps its meaning for any rubric;
- tells the model to treat the student answer as data and to ignore instructions written inside it.

Demo mode

If no `GROQ_API_KEY` is set, or `MOCK_LLM=true`, a deterministic keyword grader replaces the model so the application and the tests run offline. The interface labels this clearly as "Demo mode"; it is not a substitute for the model.

7. Accounts, roles and security

Access is role-based. The teacher account is created from the TEACHER_USERNAME and TEACHER_PASSWORD settings on every start; students create their own account.

Capability	Teacher	Student
Submit and view own reviews, download PDF	Yes	Yes (own only)
See all reviews, history filters, CSV export	Yes	No
Create or delete questions and rubrics	Yes	No
Teacher queue: approve or override	Yes	No
Batch review, re-evaluate, integrity check, dashboard	Yes	No
Similarity data and LangSmith run ids in responses	Yes	Removed

- Passwords are hashed with PBKDF2-SHA256 and a random salt.
- Sessions are signed, expiring HttpOnly cookies; failed logins are throttled (5 per minute per username).
- A student requesting another student's review receives 404; teacher endpoints return 403 for students.

- All text shown in the interface is escaped; CSV exports neutralise formula injection; uploads are limited to 5 MB and to txt, md, pdf and docx.
- The application targets local or classroom use. A public deployment should use HTTPS and strong SECRET_KEY and TEACHER_PASSWORD values.

8. Batch review, uploads and re-evaluation

- **File upload:** POST /api/extract reads text from PDF (pypdf), DOCX (python-docx), TXT and MD. Scanned PDFs without a text layer cannot be read.
- **Batch review:** the teacher uploads a CSV with student_name and answer columns, or many files (one per student). The browser reviews them one after another with a progress bar and offers a results CSV. The limit is 100 answers per batch.
- **Re-evaluate:** a teacher can re-run any review. The previous AI score is kept and shown, the earlier teacher decision is cleared and the review is routed again.

9. Integrity check and AI-versus-teacher agreement

Similarity. Each answer is lower-cased and split into words; the set of word 3-grams is compared with the Jaccard index (size of intersection divided by size of union). New answers are compared with earlier answers to the same question and flagged at 60% or more. The Integrity page lists pairs above an adjustable threshold with both answers side by side. A high value is a reason to look closer, not proof of copying.

Agreement. Of the borderline reviews the teacher decided, the agreement rate is the share approved without change; for overrides, the mean absolute difference in points is reported. The dashboard also plots AI and teacher scores for recent decisions.

10. LangSmith integration

Tracing is enabled with three environment variables: `LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY` and `LANGSMITH_PROJECT`. At start-up the application exports these (including the older LANGCHAIN_* names) so the SDK picks them up. Beyond default tracing, each review run carries:

- a run name, "Assignment review", and a unique run id that is stored with the review;
- tags: assignment-reviewer and question-<id>;
- metadata: student name, question title and grader mode;
- child spans for each graph node and for the Groq call.

When a teacher approves or overrides a score, the final score is posted to the same run as feedback under the key `teacher_final_score` (value from 0 to 1). Failure to reach LangSmith is caught and never interrupts a review. When tracing is on, prompts and answers are sent to LangSmith, so only fictional or consented submissions should be traced.

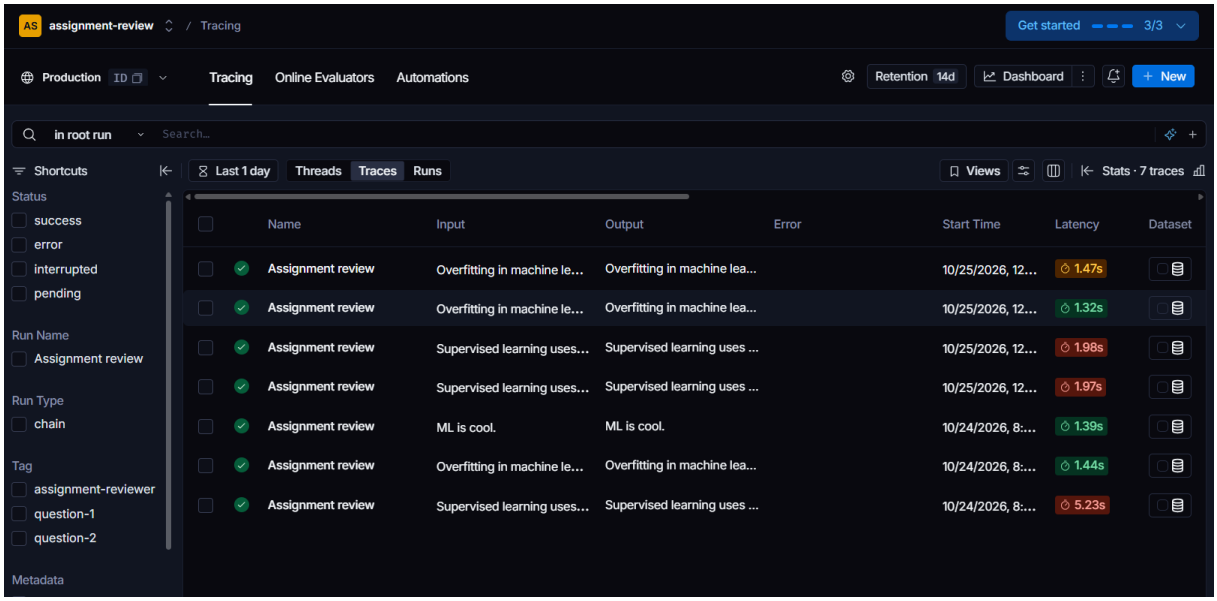


Figure: runs of the assignment-review project in LangSmith

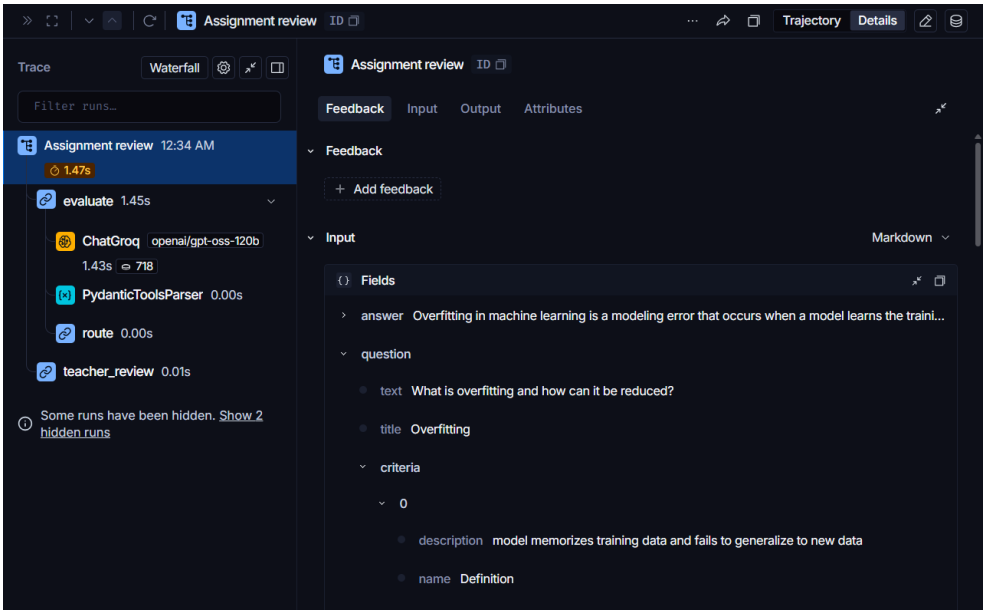


Figure: trace of one review (evaluate, ChatGroq, PydanticToolsParser, route, teacher_review)

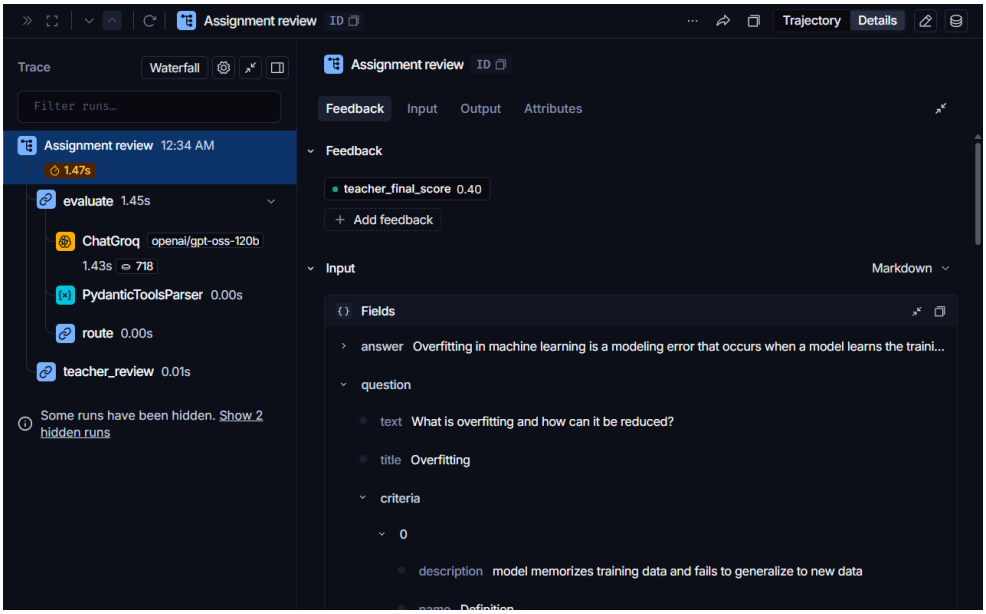


Figure: teacher decision stored on the run as teacher_final_score (0.40 means 4 out of 10)

11. REST API

Method and path	Who	Description
POST /api/auth/login, /register, /logout; GET /api/auth/me	all	Accounts and session
GET /api/health	public	Mode (demo or groq), model, tracing state, review band
GET /api/questions	login	List questions with rubrics
POST, DELETE /api/questions	teacher	Create or delete questions
POST /api/extract	login	Text from an uploaded PDF, DOCX, TXT or MD file
POST /api/reviews	login	Run the graph for an answer and store the result
GET /api/reviews (q, status, question_id, score)	login	List; students see only their own
GET /api/reviews/{id}, /pdf	login	One review or its PDF (students: own only)
DELETE /api/reviews/{id}	teacher	Delete a review
POST /api/reviews/{id}/decision	teacher	Approve or override (override needs final_score)
POST /api/reviews/{id}/reevaluate	teacher	Re-run the graph and keep the previous score
GET /api/queue	teacher	Reviews awaiting a decision
GET /api/similarity?threshold=	teacher	Pairs of similar answers
GET /api/stats	teacher	Totals, distribution, trend, criteria, agreement
GET /api/export.csv	teacher	All reviews as CSV

12. Frontend

The frontend is a dependency-free single-page application served by FastAPI. Teachers see ten views (Home, Review, Batch, History, Teacher Queue, Integrity, Dashboard, Questions, Workflow, Login); students see Home, Review, My results and Workflow. Interface features include animated score rings, confetti on a perfect score, skeleton loading, chart tooltips with click-through to the matching reviews, a Ctrl+K command palette, keyboard shortcuts, a guided tour, a print-friendly result, light and dark themes and an English or Urdu (right-to-left) interface. All text from users or the model is escaped before it is inserted into the page. The interface supports light and dark themes and adapts to narrow screens. Charts are drawn as inline SVG, so no charting library is needed.

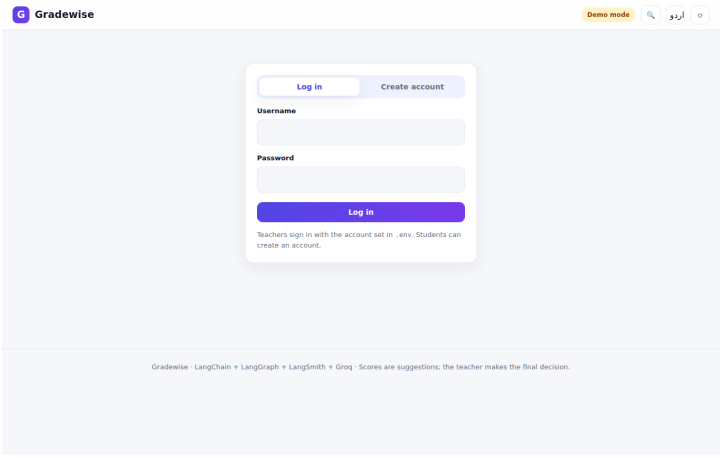


Figure 1. Login and student registration

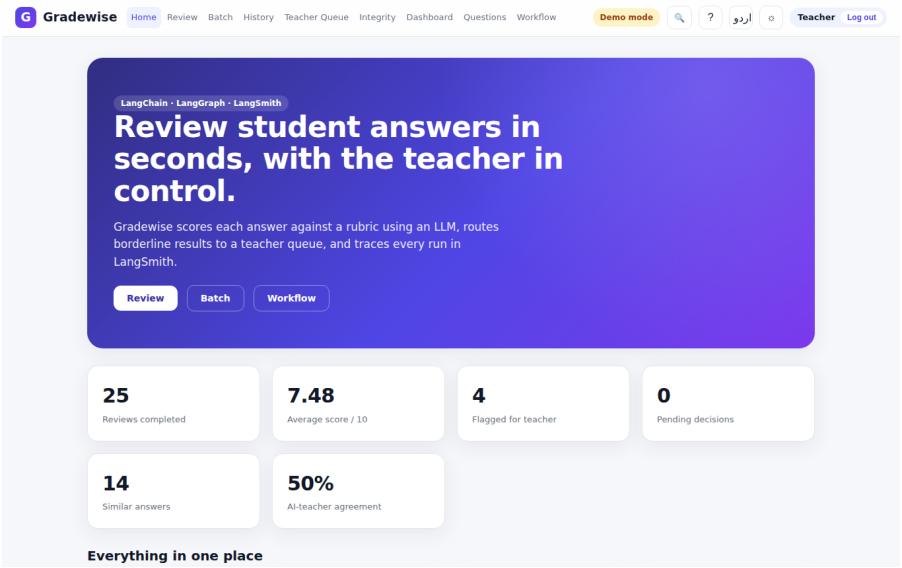


Figure 2. Home page (teacher)

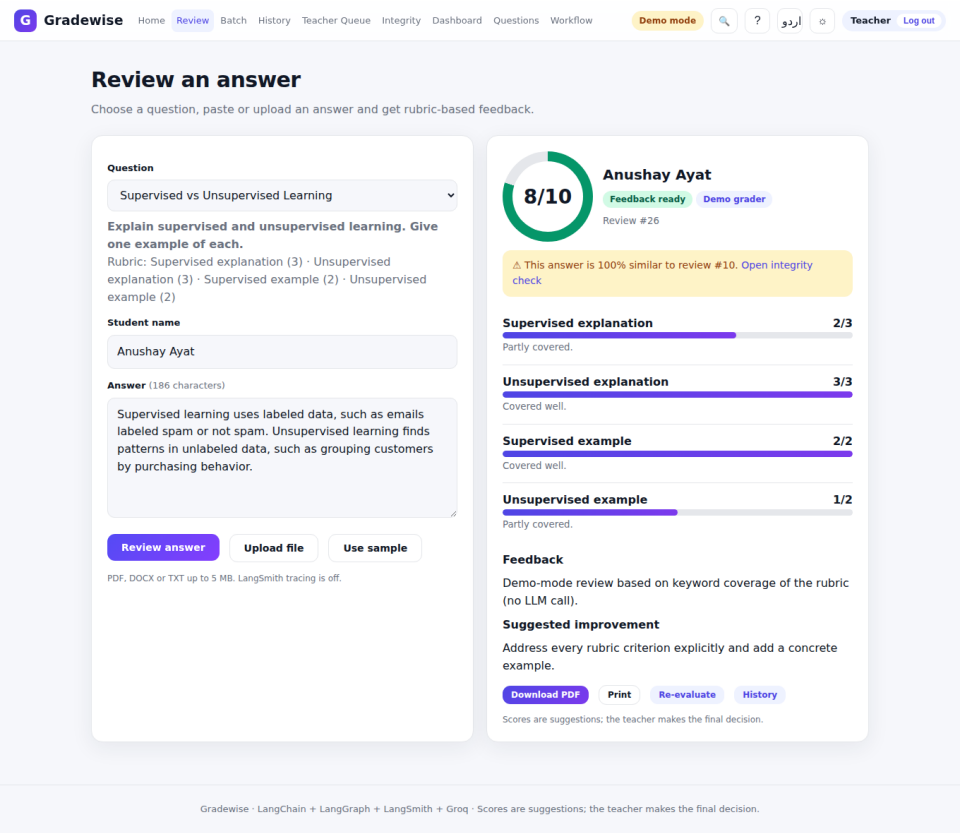


Figure 3. Review result: rubric scores, feedback, similarity warning

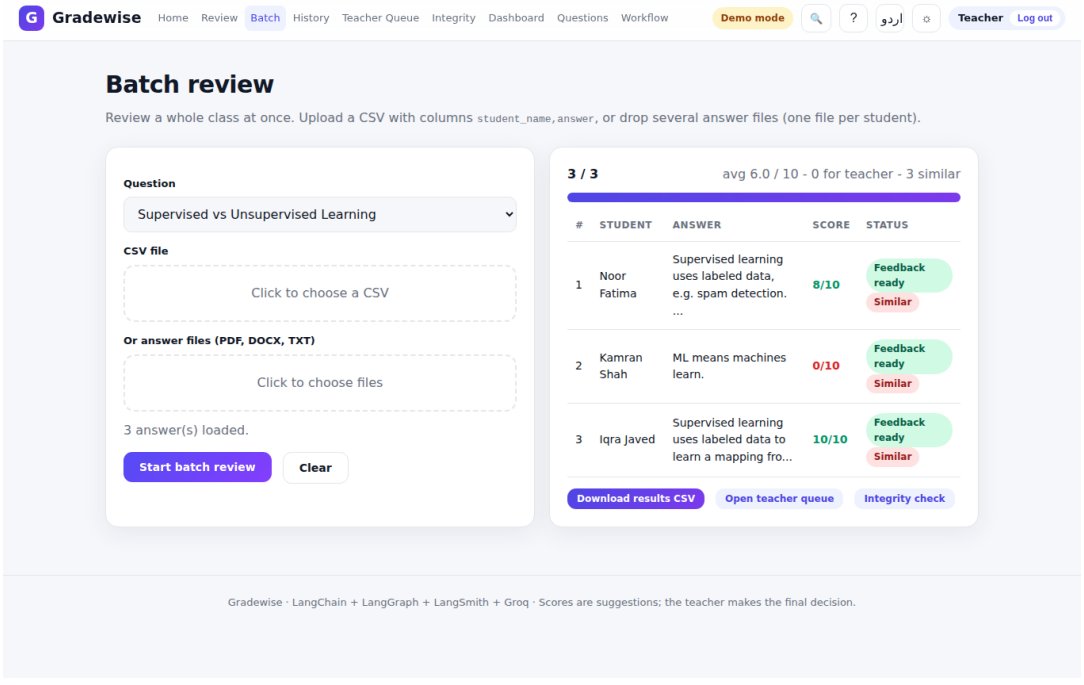


Figure 4. Batch review with live progress

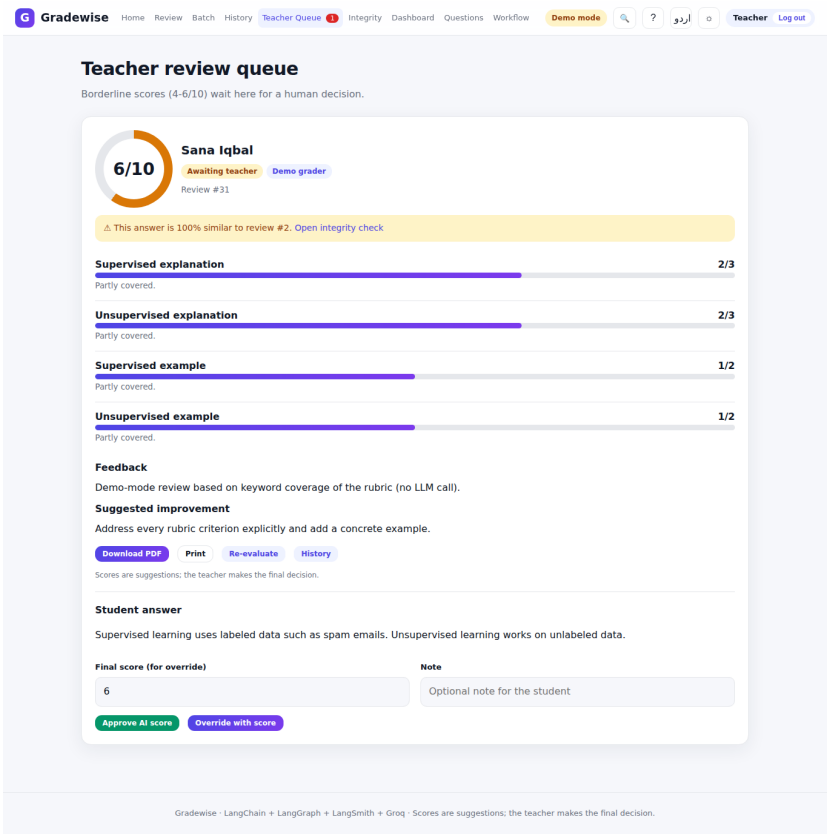


Figure 5. Teacher review queue

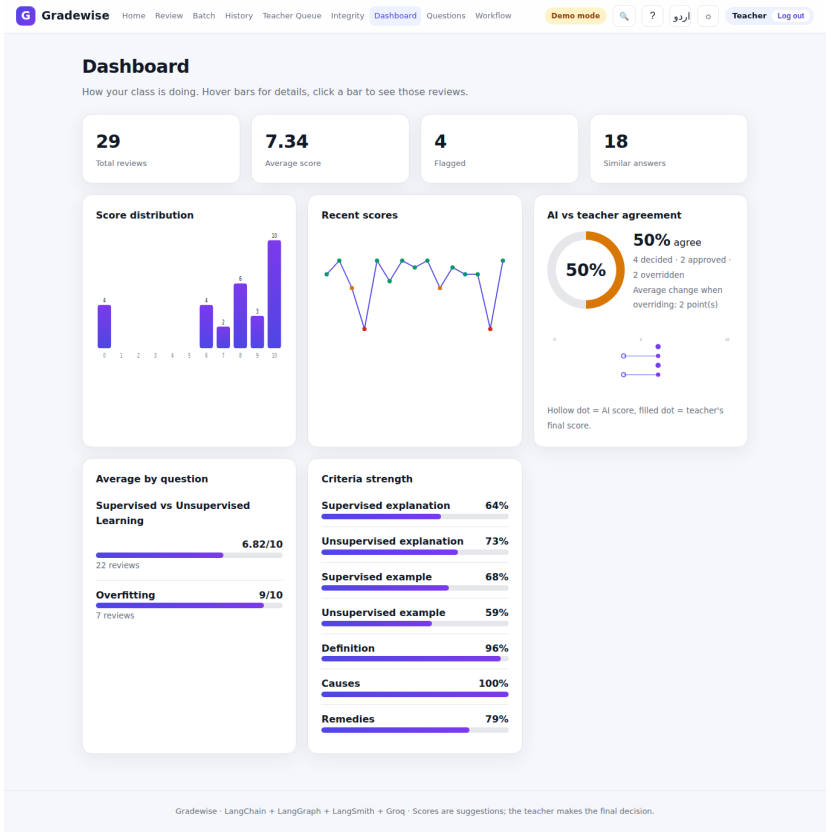


Figure 6. Dashboard with AI-versus-teacher agreement



Figure 7. Integrity check

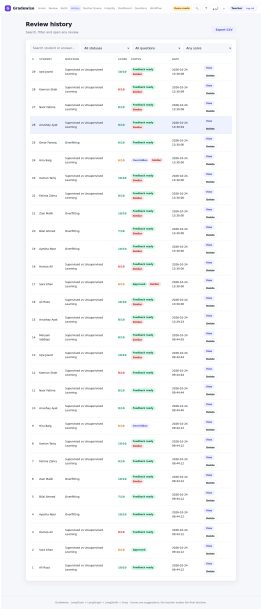


Figure 8. History with filters

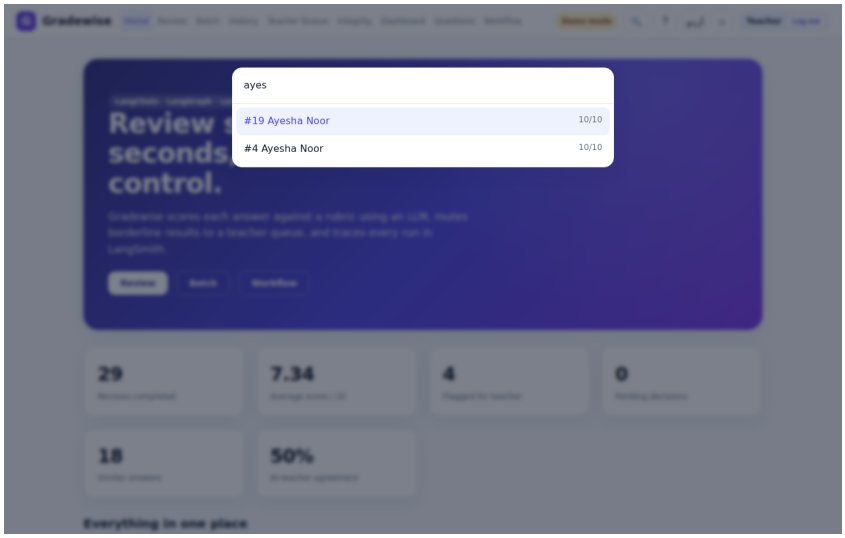


Figure 9. Command palette (Ctrl+K)

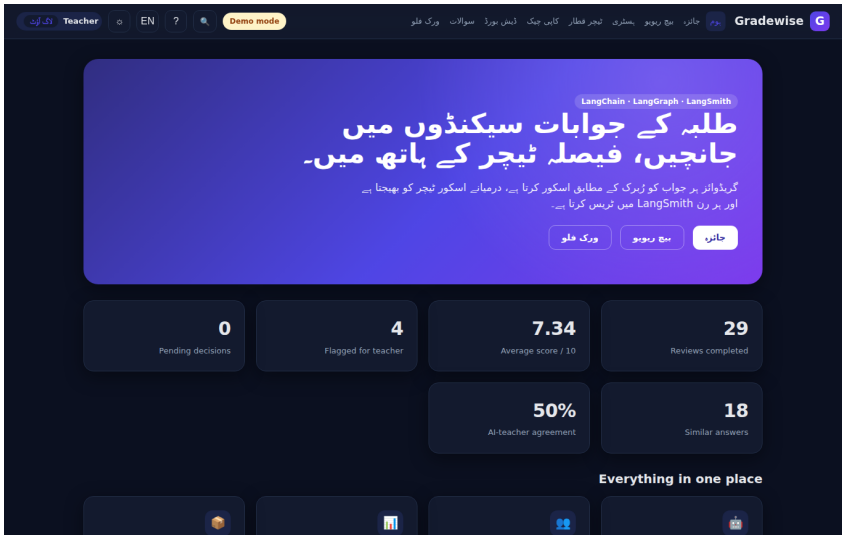


Figure 10. Urdu interface in dark mode

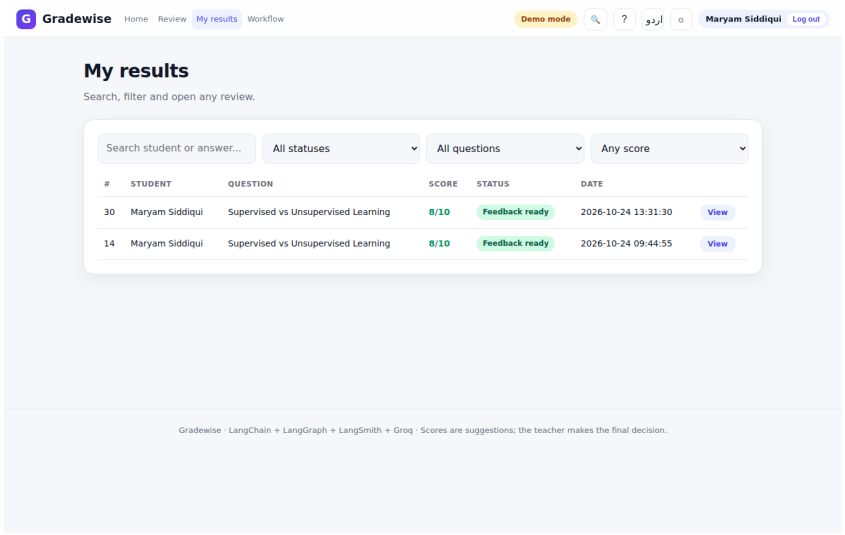


Figure 11. Student view: own results only

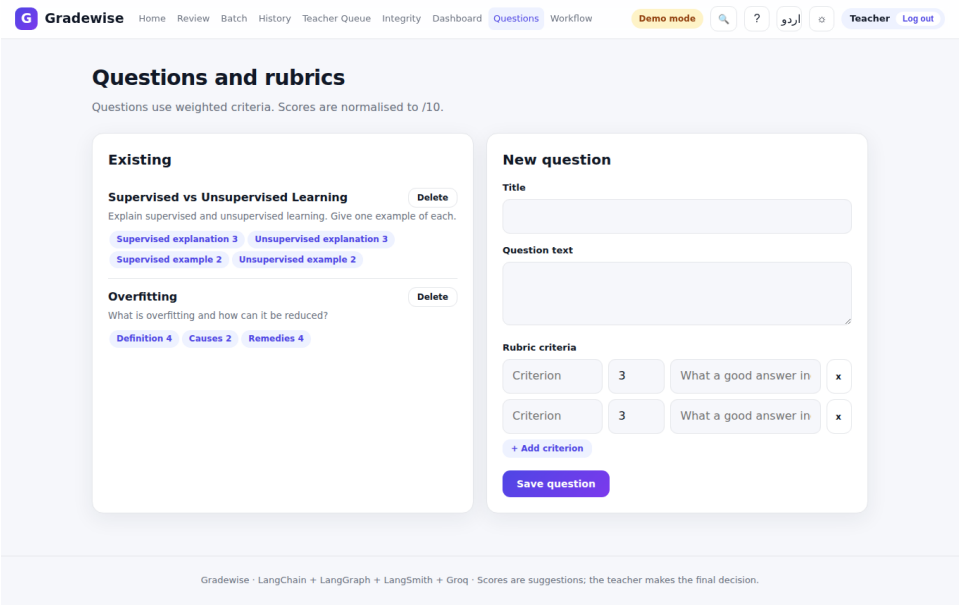


Figure 12. Custom questions and rubric builder

13. Database

Three SQLite tables are used. **users** holds username, display name, role and the password hash. **questions** holds id, title, text and the rubric as JSON. **reviews** holds the student, owner user id, answer, per-criterion scores, AI score, previous AI score, final score, feedback, improvement, status, teacher status and note, similarity and the most similar review id, grader mode and the LangSmith run id. Older databases are upgraded automatically at start-up. Two sample questions are seeded on first start.

14. Testing

Twelve automated tests run against the API in demo mode with a temporary database:

- login required for protected endpoints; wrong password rejected; health is public;
- health endpoint and question seeding;
- a complete review with every score inside its rubric range;
- routing at the band edges: 3 and 7 finish, 4, 5 and 6 go to the teacher;
- teacher override stores the final score and the PDF is valid; override without a score is rejected (422);
- custom question creation and validation;
- student isolation: name comes from the account, only own reviews are listed, teacher endpoints return 403, other reviews return 404;
- similarity detection flags an identical answer and the integrity endpoint lists the pair;
- re-evaluate keeps the previous score and clears the teacher decision;
- text extraction from TXT and DOCX, and rejection of unsupported types (415);
- statistics including agreement, score filter and CSV export.

Run them with `cd backend && python -m pytest -q`. All twelve passed during development, and the interface was exercised end to end in a headless browser (login, review, upload, batch, queue, dashboard, integrity, palette, language and student view). Live Groq and LangSmith calls were not exercised in the development environment because it had no access to those services; they require the user's own API keys.

15. How to run

```
python -m venv .venv
.venv\Scripts\activate          (Windows)   |   source .venv/bin/activate (Mac/Linux)
pip install -r requirements.txt
copy .env.example .env         (then add your keys)
python run.py
# open http://127.0.0.1:8000  -  log in as the teacher from .env, or register a student
```

16. Limitations and future work

- The AI score is a suggestion; borderline cases depend on the teacher.
- Demo mode uses keyword matching and is only for offline demonstration.
- Similarity uses wording overlap only; it does not detect paraphrased copying.
- Accounts are simple (no password reset or email); the app targets local or classroom use.
- Future work: LangSmith datasets and evaluators, OCR for scanned answers, embedding-based similarity, notifications and HTTPS deployment.

17. Conclusion

Gradewise turns the teacher's terminal exercise into a complete, observable and tested web application while keeping the original workflow logic. LangGraph provides the routing, LangChain and Groq provide the scoring, LangSmith provides traceability and a record of teacher decisions, role-based accounts protect student data, and FastAPI with a lightweight frontend makes it usable by a non-technical reviewer.